SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I G.AMRUTHA(192511250)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Amrutha

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drone for Emergency Medicine Delivery (Greedy Best-First Search vs A*)
Understanding of the Scenario
The government deploys AI-powered drones to deliver emergency medicines in flood-affected regions. The environment is dynamic because roads become blocked unexpectedly, weather changes travel cost, and the drone receives only partial real-time information. The objective is to reach the destination with **minimum delay, minimum risk, and maximum safety**.

---

i. Greedy Best-First Search (GBFS)
Working
Greedy Best-First Search selects the node with the **smallest heuristic value h(n)**, which estimates the remaining distance to the destination.
Evaluation Function

$$f(n) = h(n)$$

The algorithm ignores the cost already traveled and focuses only on reaching the goal quickly.
Strengths

- Very fast decision-making.
- Requires less computation.
- Suitable for urgent situations where immediate action is needed.
- Low memory usage.

Weaknesses

- Does not guarantee the shortest or safest route.
- May choose dangerous paths if the heuristic is misleading.
- Performs poorly when roads become blocked.

- Can get trapped in local minima.

---

ii. A* Search Algorithm

Working

A* considers both:

- Actual travel cost **g(n)**
- Estimated remaining distance **h(n)**

Evaluation Function

$$f(n) = g(n) + h(n)$$

Whenever new information about blocked roads or weather arrives, A* updates path costs and searches for a better route.

Advantages

- Finds the optimal path when the heuristic is admissible.
- Balances travel cost and estimated distance.
- Safer than Greedy Search.
- Handles varying movement costs effectively.

Disadvantages

- Requires more computation.
- Uses more memory.
- Slightly slower than Greedy Search.

---

iii. Impact of Heuristic Accuracy

Accurate Heuristic

- Greedy Search reaches the goal faster.
- A* finds the optimal route efficiently.

Inaccurate Heuristic

- Greedy Search may select blocked or risky paths.
- A* performance decreases but still considers actual travel cost, making it more reliable.

Comparison

Heuristic Accuracy	Greedy Search	A*
High	Very Fast	Fast and Optimal
Medium	May make mistakes	Mostly Optimal
Poor	Poor Decisions	Still Reliable

---

iv. Effect of Dynamic Changes

Dynamic events include:

- Flooded roads
- New obstacles
- Weather changes
- Increased travel cost

Greedy Search

- Cannot adapt efficiently.
- Often restarts the search.
- Performance degrades rapidly.

A*

- Updates path costs.
- Recalculates better routes.
- More robust in changing environments.

---

v. Recommendation

Recommended Algorithm: A*

Justification

Criterion	Reason
Real-time	Good response speed
Optimality	Finds shortest and safest path
Safety	Considers actual travel cost

Criterion	Reason
Dynamic Environment	Handles changing conditions better
Reliability	More dependable during emergencies

Conclusion

Although Greedy Search is faster, **A*** is the most suitable algorithm because emergency medicine delivery requires **safe, reliable, and near-optimal routes**, not merely the quickest decisions.

---

2. Smart City Traffic Signal Optimization

Understanding of the Scenario

The AI system controls hundreds of traffic signals to reduce:

- Total waiting time
- Fuel consumption
- Traffic congestion

The search space is extremely large, and traffic conditions continuously change.

Formulation as an Optimization Problem

Objective Function

Minimize

- Waiting Time
- Fuel Consumption
- Congestion

Constraints

- Traffic signal timing limits
- Pedestrian crossings
- Emergency vehicle priority
- Maximum queue length

Why Traditional Search is Inefficient

- Huge number of signal combinations
- Exponential search space
- Continuous changes in traffic
- Very high computational cost

---

i. Hill Climbing

Working

1. Start with current signal timings.
2. Evaluate traffic performance.
3. Modify timings slightly.
4. Accept improvements.
5. Repeat until no better solution exists.

Advantages

- Simple
- Fast
- Low memory

Limitations

- Gets stuck in local maxima
- Cannot escape plateaus
- Sensitive to initial solution

---

ii. Local Maxima, Plateaus and Ridges

Local Maximum

AI believes the current timing is best although a better global solution exists.

Example

One intersection improves while city-wide congestion increases.

Plateau

Many neighboring solutions have equal performance.

Example

Changing green time by one second produces no improvement.

Ridge
The optimal solution requires several coordinated changes.
Example
Improving traffic requires adjusting multiple nearby intersections simultaneously.

---

iii. Simulated Annealing and Genetic Algorithms
Simulated Annealing

- Occasionally accepts worse solutions.
- Escapes local maxima.
- Gradually becomes more selective.

Advantages

- Explores more solutions.
- Better than Hill Climbing.

Genetic Algorithm
Steps

1. Generate population.
2. Evaluate fitness.
3. Selection.
4. Crossover.
5. Mutation.
6. Repeat.

Advantages

- Explores many solutions simultaneously.
- Avoids local maxima.
- Highly scalable.
- Suitable for large traffic networks.

---

iv. Recommendation
Recommended Method: Genetic Algorithm
Justification

Criterion	Reason
Scalability	Handles hundreds of intersections
Adaptability	Works with changing traffic
Optimization	Finds high-quality global solutions
Parallel Search	Evaluates multiple solutions simultaneously
Robustness	Less likely to get trapped

Conclusion
The **Genetic Algorithm** is the most suitable because it provides scalable, adaptive, and near-global optimization for large smart-city traffic systems.

---

3. Autonomous Mars Rover (Online Search Agent)
Understanding of the Scenario
The Mars rover explores unknown terrain with limited sensing capability. It must safely navigate, avoid hazards, collect samples, and update its knowledge continuously.

---

i. Why Online Search?
Offline Search
Requires a complete map before planning.
Mars rover does not have one.
Online Search

- Learns while exploring.
- Makes decisions in real time.
- Updates routes continuously.

Hence, online search is required.

---

ii. Challenges
Partial Observability

- Unknown obstacles
- Limited sensors
- Hidden hazards

Dynamic Environment

- Dust storms
- Loose rocks
- Sensor uncertainty

These require continuous replanning.

---

iii. Internal Model

The rover stores:

- Explored locations
- Hazard positions
- Safe paths
- Collected samples

Whenever new information is discovered, the map is updated and future decisions improve.

---

iv. Comparison

Feature	Uninformed	Informed	Online Search
Map Required	Yes	Yes	No
Uses Heuristic	No	Yes	Optional
Learns During Search	No	No	Yes
Best for Unknown World	No	Limited	Yes

---

v. Recommendation

Recommended Approach

Online Search Agent

Justification

- Handles uncertainty.
- Learns continuously.
- Adapts to new hazards.
- Ensures safer navigation.
- Suitable for Mars exploration.

---

4. University Exam Timetabling (Constraint Satisfaction Problem)

CSP Formulation

Variables

Each course examination.

Example

- C1
- C2
- C3
- C4

Domains

Possible:

- Dates
- Time slots
- Examination halls

Constraints

1. No student has overlapping exams.
2. Hall capacity must not exceed limits.
3. Subject ordering constraints.
4. Faculty availability.
5. Special accommodation requirements.

---

i. Variables, Domains and Constraints

Component	Explanation
Variables	Exam schedules
Domains	Available time slots and halls
Constraints	Rules that every schedule must satisfy

---

ii. Backtracking Search

Working

1. Select an unscheduled exam.
2. Assign a time slot.
3. Check constraints.
4. If violated, undo assignment.
5. Try another value.
6. Continue until all exams are scheduled.

Advantages

- Always finds a valid solution if one exists.
- Systematic exploration.

---

iii. Forward Checking

After assigning one exam:

- Remove conflicting slots from remaining exams.
- Detect dead ends early.
- Reduce unnecessary search.

Benefits

- Faster search.
- Fewer backtracks.
- Improved efficiency.

---

iv. Real-World Challenges

- Thousands of students.
- Hundreds of courses.
- Limited halls.
- Faculty availability.
- Last-minute changes.

Best Strategy

Backtracking + Forward Checking + Constraint Propagation

Justification

- Efficient pruning.
- Handles large CSPs.
- Improves scalability.
- Produces feasible timetables.

---

5. Strategic Game AI (Minimax and Alpha-Beta Pruning)

Understanding of the Scenario

The AI competes against human players in a real-time strategy game. It must make strong decisions within strict time limits despite a very large decision tree.

---

i. Minimax Algorithm

Minimax assumes:

- AI tries to maximize its score.
- Opponent tries to minimize AI's score.

The algorithm evaluates future moves and selects the move with the highest guaranteed payoff.

---

ii. Computational Challenges

Problems

- Huge branching factor.
- Large search depth.
- High memory usage.
- Slow computation.

Time Complexity

$$O(b^d)$$

where:

- **b** = branching factor
- **d** = search depth

---

iii. Alpha-Beta Pruning

Alpha-Beta Pruning removes branches that cannot influence the final decision.

Benefits

- Evaluates fewer nodes.
- Produces the same optimal result as Minimax.
- Faster execution.
- Allows deeper search within time limits.

---

iv. Evaluation Function

Example Evaluation Function

$$Score = (5 \times Resources) + (4 \times ArmyStrength) + (3 \times TerritoryControl) + (2 \times UnitHealth) - (EnemyThreat)$$

Factors

- Resources
- Army strength
- Territory control
- Unit health
- Enemy threat
- Strategic position

These factors help estimate the quality of a game state when the full search cannot reach the end of the game.

---

v. Recommendation

Recommended Strategy

Minimax + Alpha-Beta Pruning + Evaluation Function

Justification

Criterion	Reason
Optimality	Same result as Minimax
Speed	Alpha-Beta prunes unnecessary branches
Real-Time Performance	Faster decision making
Scalability	Supports deeper searches
Game Strength	Better quality decisions

Conclusion

Combining **Minimax**, **Alpha-Beta Pruning**, and a well-designed **evaluation function** provides the best balance between **optimality, computational efficiency, and real-time performance**, making it ideal for strategic game AI.